PART 3

シーン間共有データ

3

シーンが変わっても覚えておきたい値を持つ

GameDataHolder.cs　（クリア状況を覚えておく）

C# Assets/Scripts/GameDataHolder.cs

```
1  using UnityEngine;
2
3  public class GameDataHolder : PersistentSingleton<GameDataHolder>
4  {
5
6      [SerializeField] public bool[] isClears = new bool[SceneData.INGAME_COUNT];
7
8      /// <summary>
9      /// ゲーム起動時に生成
10     /// </summary>
11     [RuntimeInitializeOnLoadMethod]
12     private static void Init()
13     {
14         GameDataHolder pref = Resources.Load<GameDataHolder>("Singletons/GameDataHolder");
15         if (pref != null) Instantiate(pref);
16     }
17
18     protected override void Awake()
19     {
20         base.Awake();
21     }
22 }
```

全文　1 / 4

SceneDatas.cs　（シーン名と次のシーン）

C# Assets/Scripts/SceneDatas.cs

```
1  using UnityEngine.SceneManagement;
2
3  public enum SceneName
4  {
5      Title,
6      Tutorial_0,
7      Tutorial_1,
8      Tutorial_2,
9      FirstIngame_0,
10     FirstIngame_1,
11     FirstIngame_2,
12     SecondIngame_0,
13     SecondIngame_1,
14     SecondIngame_2,
```

SceneDatas.cs （シーン名と次のシーン）

C# Assets/Scripts/SceneDatas.cs

```
    Max,
}

public class SceneData
{
    public const int INGAME_COUNT = 3;

    public static string GetSceneName(SceneName name)
    {
        switch (name)
        {
            case SceneName.Title:   return "Title";
            case SceneName.Tutorial_0: return "Tutorial_0";
            case SceneName.Tutorial_1: return "Tutorial_1";
            case SceneName.Tutorial_2: return "Tutorial_2";
            case SceneName.FirstIngame_0: return "FirstIngame_0";
            case SceneName.FirstIngame_1: return "FirstIngame_1";
            case SceneName.FirstIngame_2: return "FirstIngame_2";
            case SceneName.SecondIngame_0: return "SecondIngame_0";
            case SceneName.SecondIngame_1: return "SecondIngame_1";
            case SceneName.SecondIngame_2: return "SecondIngame_2";
```

SceneDatas.cs （シーン名と次のシーン）

C# Assets/Scripts/SceneDatas.cs

```
38          }
39
40          return "";
41      }
42
43
44      public static string GetNextScene()
45      {
46          // TODO:マジックナンバーもりもりにつき後で治す
47
48          string nowScene = SceneManager.GetActiveScene().name;
```

SceneDatas.cs　（シーン名と次のシーン）

C# Assets/Scripts/SceneDatas.cs

```
50          int lastNumber = int.Parse(nowScene[^1].ToString());
51          int nextNumber = lastNumber + 1;
52
53          string sceneNameType = nowScene.Substring(0, nowScene.Length - 2);
54          if (nextNumber == 3)
55          {
56              switch (sceneNameType)
57              {
58                  case "Tutorial": GameDataHolder.Instance.isClears[0] = true; break;
59                  case "FirstIngame": GameDataHolder.Instance.isClears[1] = true; break;
60                  case "SecondIngame": GameDataHolder.Instance.isClears[2] = true; break;
61              }
62              return GetSceneName(SceneName.Title);
63          }
64          else
65          {
66              return sceneNameType + "_" + nextNumber;
67          }
68
69      }
70  }
```

解説 ①

なぜ GameDataHolder が必要か

考え方

- シーンを切り替えると、シーン内の物は**全部破棄**される
- クリア状況をシーン内の物に持たせると、**タイトルに戻った時点で消える**
- **DontDestroyOnLoad** の物に持たせれば残る

isClears の中身

- isClears[0]：Tutorial
- isClears[1]：FirstIngame
- isClears[2]：SecondIngame
- タイトルでこれを見て、**クリア済みの印**などを出せる

POINT ゲームを終了すると消える。**次に起動しても残したい**なら、ファイルや PlayerPrefs への**保存**が別に必要。

解説 ②

シーン名を enum で管理する

C# Assets/Scripts/SceneDatas.cs

```
3   public enum SceneName
4   {
5       Title,
6       Tutorial_0,
7       Tutorial_1,
8       Tutorial_2,
9       FirstIngame_0,
10      FirstIngame_1,
11      FirstIngame_2,
12      SecondIngame_0,
13      SecondIngame_1,
14      SecondIngame_2,
15
16      Max,
17  }
```

ポイント

- 文字列の**打ち間違い**を防ぐ
- **Max**：数を数えるための目印
- GetSceneName の switch は **name.ToString()** の1行でも同じ結果になる

解説 ③

GetNextScene — 今のシーン名を分解する

C# Assets/Scripts/SceneDatas.cs

```
44      public static string GetNextScene()
45      {
46          // TODO:マジックナンバーもりもりにつき後で治す
47
48          string nowScene = SceneManager.GetActiveScene().name;
49
50          int lastNumber = int.Parse(nowScene[^1].ToString());
51          int nextNumber = lastNumber + 1;
52
53          string sceneNameType = nowScene.Substring(0, nowScene.Length - 2);
```

ポイント

- **nowScene[^1]**：最後の1文字（^1 は後ろから1番目）
- **Substring(0, 長さ - 2)**：最後の「_数字」を除いた部分

解説 ④

GetNextScene — 最後のステージならクリアを記録

C# Assets/Scripts/SceneDatas.cs

```
54          if (nextNumber == 3)
55          {
56              switch (sceneNameType)
57              {
58                  case "Tutorial": GameDataHolder.Instance.isClears[0] = true; break;
59                  case "FirstIngame": GameDataHolder.Instance.isClears[1] = true; break;
60                  case "SecondIngame": GameDataHolder.Instance.isClears[2] = true; break;
61              }
62              return GetSceneName(SceneName.Title);
63          }
64          else
65          {
66              return sceneNameType + "_" + nextNumber;
67          }
68
```

- 3つ目（_2）をクリアしたら、**GameDataHolder** に記録してタイトルへ
- それ以外は「種類 + _ + 次の番号」

確認

GetNextScene をたどってみる

今のシーン	lastNumber	nextNumber	sceneNameType	戻り値
Tutorial_0	0	1	Tutorial	Tutorial_1
FirstIngame_1	1	2	FirstIngame	FirstIngame_2
FirstIngame_2	2	3	FirstIngame	Title（isClears[1] = true）
SecondIngame_2	2	3	SecondIngame	Title（isClears[2] = true）

POINT シーン名のルール（**種類_番号**）に強く頼っている。シーン名を1つ変えるだけで動かなくなる点に注意。

PART 4

シーンに置くシングルトン

4

InGameUi と TPSCameraController

InGameUi.cs （Game Over / Game Clear 表示）

C# Assets/Scripts/InGameUi.cs

```
1  using TMPro;
2  using UnityEngine;
3  using UnityEngine.SceneManagement;
4
5  [RequireComponent(typeof(Animator))]
6  public class InGameUi : SceneSingleton<InGameUi>
7  {
8      Animator anim;
9      [SerializeField] TextMeshProUGUI tmpro;
```

InGameUi.cs （Game Over / Game Clear 表示）

C# Assets/Scripts/InGameUi.cs

```
11      bool isGameOver = false;
12      bool isGameClear = false;
13
14      protected override void Awake()
15      {
16          base.Awake(); // 追加：シングルトンの登録
17          anim = GetComponent<Animator>();
18      }
19
20      public void Update()
21      {
22          if (isGameOver)
23          {
24              // エンターキーを押した際の処理
25              if (Input.GetKeyDown(KeyCode.Return))
26              {
27                  // Uiのアニメーションをもとに戻す
28                  InGameUi.Instance.UiClear();
```

InGameUi.cs （Game Over / Game Clear 表示）

C# Assets/Scripts/InGameUi.cs

```
            //// シーンのリロード
            string sceneName = SceneManager.GetActiveScene().name;
            SceneLoadManager.Instance.LoadScene(sceneName);
        }
    }


    if (isGameClear)
    {
        // エンターキーを押した際の処理
        if (Input.GetKeyDown(KeyCode.Return))
        {
            //// Uiのアニメーションをもとに戻す
            InGameUi.Instance.UiClear();

            //// シーンのリロード
            string sceneName = SceneData.GetNextScene();
            SceneLoadManager.Instance.LoadScene(sceneName);
        }
    }
}
```

InGameUi.cs （Game Over / Game Clear 表示）

C# Assets/Scripts/InGameUi.cs

```
52      public void GameOver()
53      {
54          anim.SetTrigger("GameOver");
55          tmpro.text = "Game Over";
56          tmpro.color = Color.red;
57          isGameOver = true;
58      }
59
60      public void GameClear()
61      {
62          anim.SetTrigger("GameClear");
63          tmpro.text = "Game Clear";
64          tmpro.color = Color.white;
65          isGameClear = true;
66      }
67
68      public void UiClear()
69      {
70          anim.SetTrigger("UiClear");
71      }
72  }
```

解説 ②

Animator のトリガーで UI を動かす

C# Assets/Scripts/InGameUi.cs

```
52      public void GameOver()
53      {
54          anim.SetTrigger("GameOver");
55          tmpro.text = "Game Over";
56          tmpro.color = Color.red;
57          isGameOver = true;
58      }
59
60      public void GameClear()
61      {
62          anim.SetTrigger("GameClear");
63          tmpro.text = "Game Clear";
64          tmpro.color = Color.white;
65          isGameClear = true;
66      }
67
68      public void UiClear()
69      {
70          anim.SetTrigger("UiClear");
71      }
```

ポイント

- **SetTrigger** で Animator の遷移を起こす
- 文字と色を変えて、**isGameOver / isGameClear** を立てる

解説 ③

Game Over 中に Enter → 同じシーンをやり直す

C# Assets/Scripts/InGameUi.cs

```
22        if (isGameOver)
23        {
24            // エンターキーを押した際の処理
25            if (Input.GetKeyDown(KeyCode.Return))
26            {
27                // Uiのアニメーションをもとに戻す
28                InGameUi.Instance.UiClear();
29
30                //// シーンのリロード
31                string sceneName = SceneManager.GetActiveScene().name;
32                SceneLoadManager.Instance.LoadScene(sceneName);
33            }
34        }
```

ポイント

- **GetActiveScene().name**：今のシーン名
- 読み込みは **SceneLoadManager** に任せる

解説 ④

Game Clear 中に Enter → 次のシーンへ

C# Assets/Scripts/InGameUi.cs

```
37        if (isGameClear)
38        {
39            // エンターキーを押した際の処理
40            if (Input.GetKeyDown(KeyCode.Return))
41            {
42                //// Uiのアニメーションをもとに戻す
43                InGameUi.Instance.UiClear();
44
45                //// シーンのリロード
46                string sceneName = SceneData.GetNextScene();
47                SceneLoadManager.Instance.LoadScene(sceneName);
48            }
49        }
```

ポイント

- 次のシーン名は **SceneData.GetNextScene()** に聞く
- 最後のステージなら Title が返ってくる

全文　1 / 5

TPSCameraController.cs　（三人称カメラ）

C# Assets/Scripts/TPSCameraController.cs

```
1  using UnityEngine;
2  using System.Collections;
3
4  public class TPSCameraController : SceneSingleton<TPSCameraController>
5  {
6      [Header("ターゲット")]
7      public Transform target; // 追従対象（PlayerのTransform）
```

TPSCameraController.cs （三人称カメラ）

C# Assets/Scripts/TPSCameraController.cs

```
9       [Header("カメラ距離")]
10      public float distance = 5.0f;
11      public float height = 2.0f;
12
13      [Header("マウス操作")]
14      public float rotationSpeedX = 120f; // 横回転（マウスX）
15      public float rotationSpeedY = 80f;  // 縦回転（マウスY）
16
17      [Header("縦回転制限")]
18      public float minYAngle = -30f;
19      public float maxYAngle = 60f;
20
21      [Header("現在位置")]
22      [SerializeField] float yaw = 0f;   // 水平方向の角度（Y軸）
23      [SerializeField] float pitch = 10f; // 垂直方向の角度（X軸）
24
25      void Update()
26      {
27          if (target == null) return;
```

TPSCameraController.cs （三人称カメラ）

C# Assets/Scripts/TPSCameraController.cs

```
29          // マウス入力取得
30          float mouseX = Input.GetAxis("Mouse X");
31          float mouseY = Input.GetAxis("Mouse Y");
32
33          // カメラ回転角度の更新
34          yaw += mouseX * rotationSpeedX * Time.deltaTime;
35          pitch -= mouseY * rotationSpeedY * Time.deltaTime;
36          pitch = Mathf.Clamp(pitch, minYAngle, maxYAngle);
37
38          // 回転から方向ベクトルを作成
39          Quaternion rotation = Quaternion.Euler(pitch, yaw, 0f);
40          Vector3 offset = rotation * new Vector3(0, 0, -distance);
41          Vector3 targetPos = target.position + Vector3.up * height;
42
43          // カメラ位置と向きの更新
44          transform.position = targetPos + offset;
45          transform.LookAt(targetPos);
46      }
```

全文　4 / 5

TPSCameraController.cs　（三人称カメラ）

C# Assets/Scripts/TPSCameraController.cs

```
48      public void ShakeCam()
49      {
50          StartCoroutine(_ShakeCam());
51      }
52
53
54      private IEnumerator _ShakeCam()
55      {
56          float shakeDuration = 0.1f; // 揺れる時間
57          float shakeMagnitude = 0.1f; // 揺れの強さ
58          bool freezeTime = true; // TimeScaleでヒットストップも加える
59
60          Vector3 originalPosition;
61
62          originalPosition = transform.localPosition;
63
64          if (freezeTime)
65              Time.timeScale = 0.01f; // 疑似的に時間停止
66
67          float elapsed = 0f;
```

全文 5 / 5

TPSCameraController.cs　（三人称カメラ）

C# Assets/Scripts/TPSCameraController.cs

```
69          while (elapsed < shakeDuration)
70          {
71              float offsetX = Random.Range(-1f, 1f) * shakeMagnitude;
72              float offsetY = Random.Range(-1f, 1f) * shakeMagnitude;
73
74              transform.localPosition = originalPosition + new Vector3(offsetX, offsetY, 0);
75
76              if (freezeTime)
77                  yield return new WaitForSecondsRealtime(0.01f); // 固定時間で待つ
78              else
79                  yield return null;
80
81              elapsed += freezeTime ? 0.01f : Time.deltaTime;
82          }
83
84          transform.localPosition = originalPosition;
85
86          if (freezeTime)
87              Time.timeScale = 1f;
88      }
89  }
```

解説 ⑤

ShakeCam — 揺らしながら時間を止める

C# Assets/Scripts/TPSCameraController.cs

```
54      private IEnumerator _ShakeCam()
55      {
56          float shakeDuration = 0.1f; // 揺れる時間
57          float shakeMagnitude = 0.1f; // 揺れの強さ
58          bool freezeTime = true; // TimeScaleでヒットストップも加える
59
60          Vector3 originalPosition;
61
62          originalPosition = transform.localPosition;
63
64          if (freezeTime)
65              Time.timeScale = 0.01f; // 疑似的に時間停止
```

ポイント

- **timeScale = 0.01**：ゲーム全体がほぼ止まる（ヒットストップ）
- 待つのは **WaitForSecondsRealtime**（止めた時間に左右されない）
- だから SceneLoadManager は **unscaledDeltaTime**

解説 ⑥

カメラとプレイヤーをつなぐのは Player 側

Player.cs がしていること

- Player の **Start** で TPSCameraController.Instance.target = transform;
- カメラは**シーンに置いておく**だけでよい
- **Start** で読むので、カメラの Awake が先に済んでいる（前回の危険④を避けている）

Player から呼んでいるもの

- Player.GameOver() → **InGameUi**.Instance.GameOver() **TPSCameraController**.Instance.ShakeCam()
- Player.GameClear() → **InGameUi**.Instance.GameClear()

全体のつながり

ゲームオーバーからやり直しまで

① **Player.GameOver()**　やられた

② **InGameUi.GameOver()**　表示 ＋ **Camera.ShakeCam()**

③ **Enter** キー　→　今のシーン名を取得

④ **SceneLoadManager.LoadScene()**　フェードアウト

⑤ **LoadSceneAsync**　シーン内の物は作り直し

⑥ フェードイン　**AudioManager** の **BGM** は流れたまま

POINT

紫＝PersistentSingleton（残る）、**緑**＝SceneSingleton（シーンと一緒に作り直される）。どこから何を呼んでいるかを**説明できる**ことが今回のゴール。

NEXT

次回：ステートパターン

5

Player の動きを「状態」ごとのクラスに分ける

予告

Player は4つの「状態」を切り替えて動いている

Idle
止まっている

Walk
歩いている

Dash
Shift で走っている

Jump
空中にいる

今の状態	条件	次の状態
Idle / Walk / Dash	Space ＋ 接地している	Jump
Idle	移動キーを押した	Walk（Shift なら Dash）
Walk ⇔ Dash	Shift を押す / 離す	Dash / Walk
Walk / Dash	移動キーを離した	Idle
Jump	一度離れて、また着地した	Idle / Walk / Dash

予告

すべての状態の親クラス PlayerState

C# Assets/Scripts/Player.cs

```
179  public abstract class PlayerState
180  {
181      protected readonly Player player;
182      protected PlayerState(Player player) { this.player = player; }
183      public virtual void Enter() { }
184      public virtual void Update() { }
185      public virtual void Exit() { }
186  }
```

ポイント

- **abstract**：直接 new できない親クラス
- **Enter**：その状態に入ったとき
- **Update**：毎フレーム
- **Exit**：その状態を出るとき

予告

Walk 状態は「歩く」ことと「次はどこへ」だけを書く

C# Assets/Scripts/Player.cs

```
202  public class PlayerWalkState : PlayerState
203  {
204      public PlayerWalkState(Player player) : base(player) { }
205
206      public override void Update()
207      {
208          if (player.ShouldJump) { player.ChangeState(player.jumpState); return; }
209          if (!player.HasMoveInput) { player.ChangeState(player.idleState); return; }
210          if (player.IsDashing) { player.ChangeState(player.dashState); return; }
211          player.Walk();
212      }
213  }
```

- 条件に合えば **ChangeState** で別の状態へ移り、何もなければ歩く
- Dash や Jump のことは**それぞれのクラス**に書いてある → 状態を増やしても**他の状態を壊しにくい**